

Assignment Kit for Size Counting Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material approved for public release.
Distribution is limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Size Counting Standard

Overview

Overview

This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Size counting standard requirements	3
Example 1: Pascal size counting standard	4
Example 2: C++ size counting standard	6
Example 3: another size counting standard	8
Evaluation criteria and suggestions	10
Size counting standard template	11

Prerequisites

Prerequisite reading

- Chapter 3
-

Objectives

The objectives of Size Counting standard are to

- define the size counting standards that are appropriate for the programming language and environment that you will use
 - provide a basis for developing a coding standard
 - prepare for developing a program to count program size
-

Size counting standard requirements

Size counting standard requirements

Produce and document a standard for counting program size for the language and environment that you will use in this course.

Submit the completed standard using the format in the template on page 11 with your program 2 assignment package.

Example 1: Pascal size counting standard

Overview

The template is a simplified version of the SEI measurement framework.

- Use this template to describe important items.
- Tailor it to fit your needs or language.

We'll walk through two example size counting standards. The first example is for logical LOC for Pascal programs.

Completing the header

Complete the header as follows:

- the name you give this standard
 - the language you are using
 - your name
 - the date you produced this standard (or revision)
-

Count type

Choices are logical and physical LOC.

- Logical LOC counts language elements.
- Physical LOC counts text lines.

For this counting standard, you are counting logical LOC.

Statement type

Use this section to define how you will count various types of statements.

Consider the following:

- How are you going to count procedure declarations and function prototypes?
 - How will you count compiler directives?
 - Will you count blank lines or comments? Why or why not?
-

Clarifications

A fully operational standard generally requires many notes and comments. Use the clarification section for this purpose.

Continued on next page

Java LOC Counting Standard Template

Definition Name:	Example Pascal LOC Std.	Language:	Java
Author:	Jared Fernández González	Date:	05/11/2025

Count Type	Type	Comments
Physical/Logical	Logical	
Statement Type	Included	Comments
Executable	yes	Se cuenta cada línea de código no vacía tras remover comentarios.
Nonexecutable:		
Declarations	yes	Incluye líneas de declaración de clases, interfaces, enums, campos y firmas de métodos/constructores.
Compiler Directives	No/N.A.	No aplica en Java (#define, #include).
Comments	no	Se excluyen //, /* ... */ y /** ... */.
Blank lines	no	Se excluyen líneas en blanco o con solo espacios.
Other elements	Ver notas	Se cuentan '{' o '}' solos, anotaciones (@Override), y sentencias vacías ';' (lone ';').
Clarifications		Examples/Cases
Conteo por línea	Línea vacía o con espacios → NO cuenta.	El conteo es por línea. Se elimina primero el contenido comentado y se ignoran líneas en blanco. Si tras esta eliminación queda texto no vacío, la línea cuenta como 1 LOC lógica.
Eliminar comentarios	'int x = 0; // inicialización' → Sí cuenta (1 LOC).	Comentarios de línea ('// ...') se eliminan; comentarios de bloque ('/* ... */') y Javadoc ('/** ... */') se ignoran completamente, incluso si abarcan múltiples líneas.
Ignorar Javadoc	/* bloque\n multi-línea */ → NO cuenta en ninguna línea cubierta por el bloque.	Comentarios de línea ('// ...') se eliminan; comentarios de bloque ('/* ... */') y Javadoc ('/** ... */') se ignoran completamente, incluso si abarcan múltiples líneas.

Llaves cuentan	• 'System.out.println("http://ejemplo.com");' → Sí cuenta (el '/' en la URL dentro de la cadena no inicia comentario).	Las llaves solas ('{' o '}') cuentan como línea de código; también las sentencias vacías ';' y anotaciones en línea (por ejemplo, '@Override').
Código + comentario	'{' o '}' solos en su línea → Sí cuentan. '@Override' solo en la línea → Sí cuenta.	Si en una misma línea hay código y luego comentario de línea, se cuenta 1 LOC por esa línea (el comentario es ignorado).
No fraccionar	@Override' solo en la línea → Sí cuenta.	No se parte la línea por ';' para contar múltiples sentencias: la unidad de conteo es la línea resultante tras eliminar comentarios.
Sin directivas	;' (sentencia vacía) → Sí cuenta.	Directivas del compilador no aplican para Java y no se consideran.
Note 1		Implementación de conteo: Se recorre cada línea, se normaliza, se elimina comentario de línea y de bloque manteniendo un estado 'inBlockComment', y se suma 1 si queda contenido no vacío.
Note 2		Comentarios Javadoc se tratan como comentarios de bloque y no cuentan.
Note 3		El conteo es de LOC lógicas por línea; varias sentencias en la misma línea cuentan como 1 LOC.

Example 2: Java LOC counting standard

How many LOC?

Using the C++ LOC counting standard on page 7, how many LOC are in the following program fragment?

```
```java
public class Example {
 public static void main(String[] args) {
 int i, j;

 for (i = 0; i < 10; i++)
 for (j = 0; j < 10; j++)
 System.out.println(i + "_" + j);
 }
}
```
```

****Número de líneas de código (líneas de código lógicas en Java): 5****

Explicación:

1. Declaración del método → `public static void main(String[] args)` → 1 línea de código
2. Declaración de la variable → `int i, j;` → 1 línea de código
3. Bucle externo → `for (i = 0; i < 10; i++)` → 1 línea de código
4. Bucle interno → `for (j = 0; j < 10; j++)` → 1 línea de código
5. Instrucción ejecutable → `System.out.println(i + "\_" + j);` → 1 línea de código

Líneas excluidas:

- Las llaves de apertura y cierre `{ }` no se cuentan.
- Las líneas en blanco se ignoran.
- Este fragmento no contiene comentarios, por lo que no se realizan deducciones.

Continued on next page

